Dokumentasi API SBD Store

Daftar Endpoint

Method	Endpoint	Deskripsi	Autentikasi	Request Body	Respons Sukses	Respon s Error
POST	/user/register	Mendaftarkan pengguna baru dengan validasi regex	Tidak	`{name, username, email, phone, password}`	201 Created: user data	400, 500
POST	/user/login	Login pengguna (mengembalikan data user tanpa token)	Tidak	`{email, password}`	200 OK: user data	401, 429
POST	/auth/login	Login pengguna (mengembalikan JWT token)	Tidak	`{email, password}`	200 OK: token JWT	401, 429
PUT	/user/update	Memperbarui profil pengguna	Ya (JWT)	`{id, name, username, email, phone, password, balance}`	200 OK: updated user	400, 401, 404, 500
GET	/user/history	Mengambil riwayat transaksi pengguna	Ya (JWT)	-	200 OK: array transaksi	401, 500
GET	/user/total-spent	Mengambil total pengeluaran pengguna	Ya (JWT)	-	200 OK: total_spen t	401, 500
GET	/items	Menampilkan semua item	Tidak	-	200 OK: array item	500
POST	/transaction/create	Membuat transaksi baru	Ya (JWT)	`{user_id, item_id, quantity, description}`	201 Created: transaksi	401, 404, 500
GET	/transaction/:id	Mengambil detail transaksi	Ya (JWT)	-	200 OK: transaksi	401, 404, 500
POST	/transaction/pay/:i d	Membayar transaksi (rollback atomic)	Ya (JWT)	-	200 OK: payment success	400, 401, 404, 500
DELETE	/transaction/:id	Menghapus transaksi	Ya (JWT)	-	200 OK: deleted	401, 404, 500
GET	/reports/top-users	Mengambil peringkat pengguna berdasarkan total pengeluaran (dengan window function)	Tidak	`?limit=10`	200 OK: array users	500
GET	/reports/items-sold	Mengambil total quantity terjual dan pendapatan per item (dengan JOIN dan SUM)	Tidak	-	200 OK: array items	500
GET	/reports/monthly-s ales	Mengambil ringkasan penjualan bulanan (dengan	Tidak	`?year=2026`	200 OK: array monthly sales	500

		date_trunc dan GROUP BY)				

1. Autentikasi & Manajemen Pengguna

1.1. `POST /user/register`

Mendaftarkan pengguna baru dengan email dan password yang divalidasi regex.

Request Body:

```
{
  "name": "John Doe",
  "username": "johndoe",
  "email": "john@example.com",
  "phone": "+1-555-0100",
  "password": "SecurePass123!"
}
```

Respons Sukses (201 Created):

```
{
  "success": true,
  "message": "User registered successfully",
  "payload": {
    "id": 1,
    "name": "John Doe",
    "username": "johndoe",
    "email": "john@example.com",
    "phone": "+1-555-0100",
    "balance": 0
  }
}
```

Respons Error:

- **400 Bad Request** – Format email atau password tidak valid.
- **500 Internal Server Error** – Error database.

1.2. `POST /user/login`

Mengautentikasi pengguna dengan email dan password (mengembalikan data pengguna tanpa token).

Request Body:

```
{
  "email": "john@example.com",
  "password": "SecurePass123!"
}
```

Respons Sukses (200 OK):

```
{
  "success": true,
  "message": "Login successful",
  "payload": {
    "id": 1,
    "name": "John Doe",
    "email": "john@example.com",
    "balance": 1000
  }
}
```

Respons Error:

- **401 Unauthorized** – Email atau password tidak valid.
- **429 Too Many Requests** – Batas request terlampaui.

1.3. `POST /auth/login`

Mengautentikasi pengguna dan mengembalikan token JWT.

Request Body:

```
{
  "email": "john@example.com",
  "password": "SecurePass123!"
}
```

Respons Sukses (200 OK):

```
{
  "success": true,
  "message": "Login successful",
  "payload": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "user": {
      "id": 1,
      "name": "John Doe",
      "email": "john@example.com",
      "balance": 1000
    }
  }
}
```

Respons Error:

- **401 Unauthorized** – Email atau password tidak valid.
- **429 Too Many Requests** – Batas request terlampaui.

1.4. `PUT /user/update`

Memperbarui profil pengguna (memerlukan autentikasi). Password di-hash, email dan password divalidasi dengan regex.

Header: `Authorization: Bearer <token>` **Request Body:**

```
{
  "id": 1,
  "name": "John Updated",
  "username": "johnupdated",
  "email": "john.updated@example.com",
  "phone": "+1-555-0101",
  "password": "NewSecurePass456!",
  "balance": 1000
}
```

Respons Sukses (200 OK):

```
{
  "success": true,
  "message": "User updated successfully",
  "payload": {
    "id": 1,
    "name": "John Updated",
    "username": "johnupdated",
    "email": "john.updated@example.com",
    "phone": "+1-555-0101",
    "balance": 1000
  }
}
```

Respons Error:

- **400 Bad Request** – Error validasi.
- **401 Unauthorized** – Token hilang atau tidak valid.
- **404 Not Found** – Pengguna tidak ditemukan.
- **500 Internal Server Error** – Error database.

1.5. `GET /user/history`

Mengambil riwayat transaksi untuk pengguna yang diautentikasi beserta detail item.

Header: `Authorization: Bearer <token>` **Respons Sukses (200 OK):**

```
{
  "success": true,
  "message": "Transaction history retrieved",
  "payload": [
    {
      "id": 5,
      "item_name": "Laptop",
      "quantity": 1,
      "total": 1000000,
      "status": "paid",
      "created_at": "2024-03-25T10:30:00.000Z"
    },
```

```
      {
        "id": 3,
        "item_name": "Mouse",
        "quantity": 2,
        "total": 100000,
        "status": "pending",
        "created_at": "2024-03-24T09:15:00.000Z"
      }
    ]
  }
```

Respons Error:

- **401 Unauthorized** – Token hilang atau tidak valid.
- **500 Internal Server Error** – Error database.

1.6. `GET /user/total-spent`

Mendapatkan total jumlah yang dibelanjakan oleh pengguna yang diautentikasi (jumlah dari transaksi berstatus paid).

Header: `Authorization: Bearer <token>` **Respons Sukses (200 OK):**

```
  {
    "success": true,
    "message": "Total spent retrieved",
    "payload": {
      "total_spent": 1100000
    }
  }
```

Respons Error:

- **401 Unauthorized** – Token hilang atau tidak valid.
- **500 Internal Server Error** – Error database.

2. Item

2.1. `GET /items`

Menampilkan semua item yang tersedia (endpoint publik).

Respons Sukses (200 OK):

```
  {
    "success": true,
    "message": "Items retrieved successfully",
    "payload": [
      {
        "id": 1,
        "name": "Laptop",
        "price": 1000000,
        "stock": 10,
        "created_at": "2024-03-20T00:00:00.000Z"
```

```
    },
    {
      "id": 2,
      "name": "Mouse",
      "price": 50000,
      "stock": 100,
      "created_at": "2024-03-20T00:00:00.000Z"
    }
  ]
}
```

Respons Error:

- **500 Internal Server Error** – Error database.

3. Transaksi

3.1. `POST /transaction/create`

Membuat transaksi baru (status default `pending`).

Header: `Authorization: Bearer <token>` **Request Body:**

```
{
  "user_id": 1,
  "item_id": 2,
  "quantity": 3,
  "description": "Beli laptop untuk kerja"
}
```

Respons Sukses (201 Created):

```
{
  "success": true,
  "message": "Transaction created successfully",
  "payload": {
    "id": 6,
    "user_id": 1,
    "item_id": 2,
    "quantity": 3,
    "total": 150000,
    "status": "pending",
    "description": "Beli laptop untuk kerja",
    "created_at": "2024-03-25T12:00:00.000Z"
  }
}
```

Respons Error:

- **401 Unauthorized** – Token hilang atau tidak valid.
- **404 Not Found** – Item tidak ditemukan.
- **500 Internal Server Error** – Error database.

3.2. `GET /transaction/:id

Mengambil satu transaksi berdasarkan ID.

Header: `Authorization: Bearer <token>` **Respons Sukses (200 OK):**

```
{
  "success": true,
  "message": "Transaction retrieved successfully",
  "payload": {
    "id": 6,
    "user_id": 1,
    "item_id": 2,
    "quantity": 3,
    "total": 150000,
    "status": "pending",
    "description": "Beli laptop untuk kerja",
    "created_at": "2024-03-25T12:00:00.000Z"
  }
}
```

Respons Error:

- **401 Unauthorized** – Token hilang atau tidak valid.
- **404 Not Found** – Transaksi tidak ditemukan.
- **500 Internal Server Error** – Error database.

3.3. `POST /transaction/pay/:id` *(Terlindungi)*

Membayar transaksi pending. Mengurangi total dari saldo pengguna, memperbarui status menjadi `paid`. Menggunakan transaksi database untuk rollback jika gagal.

Header: `Authorization: Bearer <token>` **Respons Sukses (200 OK):**

```
{
  "success": true,
  "message": "Payment successful",
  "payload": {
    "transaction_id": 6,
    "new_balance": 850000,
    "status": "paid"
  }
}
```

Respons Error:

- **400 Bad Request** – Saldo tidak cukup, jumlah tidak valid, atau transaksi tidak pending.
- **401 Unauthorized** – Token hilang atau tidak valid.
- **404 Not Found** – Transaksi tidak ditemukan.
- **500 Internal Server Error** – Error database (rollback dilakukan).

3.4. `DELETE /transaction/:id` *(Terlindungi)*

Menghapus transaksi berdasarkan ID.

Header: `Authorization: Bearer <token>` **Respons Sukses (200 OK):**

```
{
  "success": true,
  "message": "Transaction deleted successfully",
  "payload": null
}
```

Respons Error:

- **401 Unauthorized** – Token hilang atau tidak valid.
- **404 Not Found** – Transaksi tidak ditemukan.
- **500 Internal Server Error** – Error database.

4. Respons Error

Semua respons error mengikuti struktur JSON yang sama:

```
{
  "success": false,
  "message": "Pesan error yang dapat dibaca manusia",
  "payload": null
}
```

Kode HTTP Umum:

- **400** – Bad Request (error validasi, field hilang, saldo tidak cukup).
- **401** – Unauthorized (token hilang/tidak valid, kredensial salah).
- **403** – Forbidden (opsional: jika Anda menambahkan akses berbasis peran).
- **404** – Not Found (resource tidak ada).
- **429** – Too Many Requests (batas rate terlampaui).
- **500** – Internal Server Error (error database, error server tak terduga).

5. Skema Database

5.1. `users`

Kolom	Tipe	Constraints
id	SERIAL	PRIMARY KEY
name	VARCHAR(100)	NOT NULL
username	VARCHAR(50)	UNIQUE NOT NULL
email	VARCHAR(150)	UNIQUE NOT NULL
phone	VARCHAR(20)	NULL
password	VARCHAR(255)	NOT NULL (bcrypt hash)
balance	INTEGER	DEFAULT 0
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

5.2. `items`

Kolom	Tipe	Constraints
id	SERIAL	PRIMARY KEY
name	VARCHAR(200)	NOT NULL
price	INTEGER	NOT NULL
stock	INTEGER	NOT NULL DEFAULT 0
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

5.3. `transactions`

Kolom	Tipe	Constraints
id	SERIAL	PRIMARY KEY
user_id	INTEGER	REFERENCES users(id) ON DELETE CASCADE
item_id	INTEGER	REFERENCES items(id) ON DELETE CASCADE
quantity	INTEGER	NOT NULL
total	INTEGER	NOT NULL
status	VARCHAR(20)	DEFAULT 'pending' (atau ENUM)
description	TEXT	NULL
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Catatan:

- `status` bisa berupa tipe ENUM (`pending`, `paid`) untuk batasan yang lebih ketat.
- Tambahkan indeks pada `user_id` dan `item_id` untuk JOIN yang lebih cepat.
- Pertimbangkan menambahkan timestamp `updated_at` untuk audit.

6. Endpoint Laporan

6.1. `GET /reports/top-users`

Mengambil peringkat pengguna berdasarkan total pengeluaran (hanya transaksi berstatus 'paid').

Parameter Query:

- `limit` (opsional, default: 10) – Jumlah pengguna teratas yang ditampilkan.

Respons Sukses (200 OK):

```
{
  "success": true,
  "message": "Top users retrieved successfully",
  "payload": [
    {
      "id": 1,
      "name": "John Doe",
      "username": "johndoe",
      "email": "john@example.com",
      "phone": "+1-555-0100",
      "balance": 1000,
      "total_spent": 1100000,
      "rank": 1
    }
  ]
```

```
}
```

Respons Error:

- **500 Internal Server Error** – Error database.

6.2. `GET /reports/items-sold`

Mengambil total quantity terjual dan total pendapatan per item (hanya transaksi berstatus 'paid').

Respons Sukses (200 OK):

```
{
  "success": true,
  "message": "Items sold retrieved successfully",
  "payload": [
    {
      "id": 1,
      "name": "Laptop",
      "price": 1000000,
      "stock": 10,
      "total_quantity_sold": 5,
      "total_revenue": 5000000
    }
  ]
}
```

Respons Error:

- **500 Internal Server Error** – Error database.

6.3. `GET /reports/monthly-sales`

Mengambil ringkasan penjualan bulanan (hanya transaksi berstatus 'paid').

Parameter Query:

- `year` (opsional, default: tahun saat ini) – Tahun yang ingin dianalisis.

Respons Sukses (200 OK):

```
{
  "success": true,
  "message": "Monthly sales retrieved successfully",
  "payload": [
    {
      "month": "2026-03-01T00:00:00.000Z",
      "transaction_count": 10,
      "total_revenue": 2500000
    }
  ]
}
```

Respons Error:

- **500 Internal Server Error** – Error database.